

Relatório: Projeto e Síntese do algoritmo CORDIC

Gabriel João Martins
Engenharia Eletrônica
UFSC

Santa Catarina, Brasil
gabjomar@gmail.com

Lucas Soares Ramos
Engenharia Eletrônica
UFSC

Santa Catarina, Brasil
lucas.s.ramos@grad.ufsc.br

Luiz Fernando Mina Gonzaga da
Silva

Ciências da Computação
UFSC

Santa Catarina, Brasil
luizfernandominagds@gmail.com

Nathan Santana Schunk
Ciências da Computação
UFSC

Santa Catarina, Brasil
nathanschunk@hotmail.com

Thiago Ferreira de Castro
Engenharia Eletrônica
UFSC

Santa Catarina, Brasil
thiagofcastro124@gmail.com

Abstract—Este relatório é referente ao projeto de síntese do método base do CORDIC (COordinate Rotation Digital Computer), o qual se refere a um algoritmo eficiente para o cálculo de funções trigonométricas usando apenas somas, subtrações e deslocamentos de bits (sem a utilização de multiplicações). De forma mais concreta, o CORDIC é um método iterativo que converte rotações e transformações de coordenadas em operações muito simples a nível de hardware (e de software), o que a torna mais conveniente do que alternativas que utilizam pontos flutuantes. Há implementações mais generalistas que estendem as funções trigonométricas para hiperbólicas, exponenciais e logarítmicas, porém o projeto se ateve à implementação base do algoritmo, que é capaz de calcular $\sin \theta$ e $\cos \theta$ a partir de um dado ângulo θ e $\arctan(\frac{y}{x})$ a partir de uma coordenada (x, y) , onde $-\frac{\pi}{2} \leq \arctan(\frac{y}{x}) \leq \frac{\pi}{2}$. A definição inicial do projeto se deu através do estudo do algoritmo em si, a partir da definição do comportamento, um roteiro descrevendo os procedimentos realizados foi descrito; por fim, para fins de acurácia, um protótipo do projeto foi criado em Python (tentando simular ao máximo as manipulações de caráter *low-level*), e confirmou-se que de fato o algoritmo estudado está correto. Os diagramas de estados e dos circuitos foram definidos e utilizados como base para sintetizar o projeto em VHDL.

Index Terms—Scientific writing, Logic Circuits, CORDIC, VHDL

I. INTRODUÇÃO

O CORDIC (COordinate Rotation Digital Computer) é um algoritmo iterativo que utiliza rotações para calcular um amplo número de funções elementares de forma eficiente e barata no hardware, utilizando apenas somas, subtrações e shifts para realizar esses cálculos, deixando de utilizar multiplicações ou pontos flutuantes, como no caso das funções seno e cosseno, sendo portanto, o objetivo do projeto estudar o algoritmo, apresentar suas funcionalidades e realizar uma implementação básica dele no hardware por meio de VHDL. [1]

II. PROPOSTA

A arquitetura do sistema foi dividida em dois blocos principais conectados por uma entidade de nível superior (Top-Level).

- 1) **Bloco de Controle (BC)**: Responsável pela lógica de decisão e sequenciamento dos estados.
- 2) **Bloco Operativo (Datapath)**: Responsável pelo armazenamento e manipulação aritmética dos dados.

O fluxo de dados adota uma estratégia de precisão mista. As entradas são recebidas em formato de ponto fixo Q4.5 (9 bits) para reduzir a largura de banda de entrada. Internamente, esses dados são expandidos para Q5.16 (21 bits), garantindo maior precisão durante as iterações e minimizando erros de arredondamento no resultado final.

A. Bloco Operacional (Datapath)

O “Datapath” contém a lógica aritmética e de armazenamento. Diferente de implementações que utilizam multiplicadores dedicados, esta proposta utiliza apenas deslocadores e somadores/subtratores. Os principais componentes são:

- **Conversão de Sinal**: Módulos *q45_to_q516* expandem as entradas de 9 bits para 21 bits com extensão de sinal, garantindo a precisão necessária nos estágios intermediários de cálculo.
- **Lógica Aritmética**: Três ULAs realizam as operações $x \pm (y \gg i)$, $y \pm (x \gg i)$ e $Z \pm \arctan(2^{-i})$. A direção da operação (soma ou subtração) é ditada pelo sinal *op_signal* vindo da FSM.
- **Correção de Escala Combinacional**: A multiplicação pela constante $K \approx 0.607$ (necessária no modo Vetorização) foi implementada como um circuito lógico combinacional. Este bloco realiza a operação $x_{\text{ajustado}} \approx (x \gg 1) + (x \gg 3) - (x \gg 6) - (x \gg 9) - (x \gg 12)$, atualizando o valor de x a cada iteração sem gastar ciclos de clock adicionais.
- **Registradores de Saída**: Para garantir a estabilidade dos dados finais, registradores dedicados armazenam o resultado da última iteração, isolando as saídas das flutuações naturais do processo iterativo.

B. Máquina de Estados (FSM)

O controle é exercido por uma Máquina de Estados Finitos composta por 9 estados (S0 a S7 e S9), descrita no arquivo *cordic_bc.vhdl*. A lógica foi otimizada para remover estados de processamento redundantes:

- **S0**: Estado de espera.
- **S1**: Carrega os dados e decide o fluxo: *mode* = 1 inicia a Rotação (vai para S2); *mode* = 0 inicia a Vetorização (vai para S5).
- **S2-S4**: Loop de iteração que verifica o sinal de *z* e executa as rotações até completar 16 ciclos.
- **S5-S7**: Loop de iteração que verifica o sinal de *y*. A correção da magnitude pelo ganho *K* é aplicada combinacionalmente no Datapath, permitindo que o fluxo convirja diretamente para o final.
- **S9**: Habilita a escrita nos registradores de saída e sinaliza o término.

C. Sinais de Entrada e Saída do Toplevel

A interface da entidade *toplevel* é definida conforme a listagem abaixo:

- Entradas:
 - *clock, reset*: Controle global.
 - *start* (1 bit): Pulso de início.
 - *mode* (1 bit): Seleção de operação ('1': Rotação, '0': Vetorização).
 - *x_in, y_in, z_in* (9 bits): Dados de entrada em Q4.5.
- Saídas:
 - *x_out, y_out, z_out* (21 bits): Resultados em Q5.16.
 - *done* (1 bit): Sinal de dado válido.

III. PLANO DE VERIFICAÇÃO

O funcionamento do código foi verificado por meio da realização de testbenches, utilizando GHDL e GTKWave. Dessa forma, é possível analisar se o bloco de controle, o datapath e as interações entre esses blocos estão operando corretamente, de acordo com o que foi definido nos diagramas. Além disso, é possível testar os dois modos de operação — Rotação e Vetorização — aplicando valores às entradas do CORDIC e verificando se suas saídas convergem para os valores esperados.

A. Testbench do modo Vetorização

Para o modo Vetorização, o CORDIC deve zerar o valor de *Y* ao mesmo tempo em que acumula, no registrador *Z*, o valor do ângulo correspondente a $\arctan(y/x)$. A saída *X* deve convergir para o módulo do vetor de entrada.

O testbench inicia a simulação com as seguintes entradas em Q4.5:

- *x_in* = 96
- *y_in* = 128
- *z_in* = 0 (irrelevante no modo Vetorização)

Que convertidos para valores reais, correspondem a:

- *x_in* = 3.0
- *y_in* = 4.0
- *z_in* = 0.0

Após 16 iterações do algoritmo foram observados os seguintes resultados nas saídas (em Q5.16):

- *x_out* = 327643
- *y_out* = 0
- *z_out* = 60772

Ao multiplicar esses valores por 2^{-16} obtém-se os dados em números decimais reais; correspondem a:

- *x_out* = 4.9994354248046875
- *y_out* = 0.0
- *z_out* = 0.92730712890625

A interpretação desses valores se dá da seguinte forma: *x_out* converge para a hipotenusa do triângulo formado por catetos de extensão 3 e 4. O valor calculado $\approx 4,9994$ é muito próximo de 5, confirmando o funcionamento correto do algoritmo. *y_out*, como esperado, converge para 0, comportamento característico do modo Vetorização. *z_out* converge para $\arctan(\frac{4}{3}) \approx 0,9273$ rad, equivalente a aproximadamente $53,13^\circ$.

B. Testbench do modo Rotação

No modo Rotação, o CORDIC deve aplicar a Rotação definida por um ângulo *z_in* ao vetor de entrada (*x_in, y_in*). Após a Rotação, o valor de *z* deve convergir para zero, enquanto *x* e *y* convergem para $\cos(z_{in})$ e $\sin(z_{in})$, respectivamente.

O testbench inicia a simulação com as seguintes entradas em Q4.5:

- *x_in* = 0 (irrelevante no modo Rotação)
- *y_in* = 0 (irrelevante no modo Rotação)
- *z_in* = 25

Que convertidos para valores reais, correspondem a:

- *x_in* = 0.0
- *y_in* = 0.0
- *z_in* = 0.78125 rad $\approx 45^\circ$

Após 16 iterações do algoritmo foram observados os seguintes resultados nas saídas (em Q5.16):

- *x_out* = 46533
- *y_out* = 46146
- *z_out* = 0

Ao multiplicar esses valores por 2^{-16} obtém-se os dados em números decimais reais; correspondem a:

- *x_out* = 0.7100372314453125
- *y_out* = 0.704132080078125
- *z_out* = 0.0

Interpretando esses valores, tem-se que:

- $x_{out} \approx \cos(45^\circ) \approx 0.7100 \approx \frac{\sqrt{2}}{2}$
- $y_{out} \approx \sin(45^\circ) \approx 0.7041 \approx \frac{\sqrt{2}}{2}$
- $z_{out} = 0$

C. Sobre os resultados dos testbenches

Conclusão, os resultados obtidos nos testbenches confirmam que a arquitetura implementada do CORDIC está funcionando corretamente em ambos os modos de operação. As saídas convergem para os valores matematicamente esperados, validando tanto o bloco de controle quanto o datapath da implementação.

Um detalhe relevante a se traçar é: a quantidade de bits das entradas dedicados aos valores decimais após a vírgula

determina o quão preciso o resultado final será, caso a entrada estivesse em Q4.16 (4 bits para a parte inteira, 16 bits para a parte decimal), o resultado seria extremamente mais preciso. Por exemplo, enquanto o valor 3 em Q4.5 é definido por 96 ($3 \cdot 2^5$), em Q4.16 a representação seria dada por 196608 ($3 \cdot 2^{16}$).

D. Resultados de síntese

A validação física do projeto foi realizada através da ferramenta de síntese, visando confirmar a viabilidade da implementação do algoritmo CORDIC na arquitetura de precisão mista proposta. Os resultados abaixo detalham o ambiente de desenvolvimento, a ocupação de área no dispositivo e a análise de temporização.

a) Ambiente de desenvolvimento e dispositivo alvo:

A síntese foi executada no software Quartus Prime (Versão 20.1.0), rodando sob o sistema operacional Arch Linux. O dispositivo FPGA selecionado para a implementação pertence à família Cyclone IV GX, adequada para aplicações que exigem um equilíbrio entre custo e performance lógica.

- Família: Cyclone IV GX
- Dispositivo: EP4CGX150DF31C7

b) Utilização de recursos:

A arquitetura proposta, que evita o uso de multiplicadores dedicados em favor de deslocadores e somadores, resultou em uma ocupação eficiente dos recursos lógicos do FPGA.

1) **Total Combinational Functions:** 478

Este valor reflete a complexidade da “Lógica Aritmética” do Datapath, composta pelas três ULAs que realizam as operações de soma/subtração e deslocamento barrel-shifter 2, bem como a lógica de correção de escala combinacional ($K \approx 0.607$).

2) **Dedicated Logic Registers:** 160

O uso de registradores deve-se principalmente aos:

- Registradores de Saída dedicados (21 bits cada para x_{out} , y_{out} , z_{out}) que garantem a estabilidade dos dados finais.
- Registradores internos de estado para as variáveis x , y , z durante o processo iterativo.
- Máquina de Estados Finitos (FSM) de 9 estados, responsável pelo sequenciamento do controle.

3) **Total Pins:** 95

O número de pinos utilizados (95) corresponde exatamente à especificação da interface do toplevel:

- Entradas (31 pinos): *clock*, *reset*, *start*, *mode* e os vetores de entrada Q4.5 (x_{in} , y_{in} , z_{in}).
- Saídas (64 pinos): Sinal *done* e os vetores de resultado expandidos em Q5.16 (x_{out} , y_{out} , z_{out}).

IV. CONCLUSÃO

É evidenciado, portanto, por meio do objeto de estudo e da implementação bem-sucedida do CORDIC via VHDL, as características básicas e os benefícios de se utilizar o algoritmo para realizar o cálculo de funções, que mesmo possuindo uma precisão de resultado possivelmente menor do que outros métodos, se não for calculado um número suficiente de

iterações, se torna uma opção relevante pelo menor custo de implementação e maior eficiência na execução, sendo válida para contextos onde a precisão de valores não é primordial para a criação de um sistema ou dispositivo, como por exemplo, o caso de processamento de sinais, de imagem e no cálculo de movimentos de manipuladores robóticos seriais.

REFERENCES

- [1] P. K. E. a. MEHER, “50 years of CORDIC: Algorithms, architectures, and applications. IEEE Transactions on Circuits and Systems I: Regular Papers,” vol. 56, pp. 1893–1907, 2009.